# DNN Programming Assignment-2: Report

**Name:** Manish Raj | **Student ID:** 2026AJ05499

*Variant: seeds 11, 22, 33 | hidden size 64 | student-ID last digit 9 (odd: LSTM forget-gate bias ablation) | PyTorch, GPU (CUDA)*

## 1. Problem 1 - recurrent models

### 1.1 Setup: datasets, split, preprocessing

**Problem 1 (recurrent models).** The RNN, LSTM and GRU share the same optimisation recipe so that differences in the results come from the cell and not from tuning: Adam with learning rate 1e-3, batch size 128, gradient-norm clipping at 1.0 and an equal number of epochs per task (20 for UCI HAR, 30 for the adding problem). The epoch with the best validation score (macro-F1 for HAR, MSE for the adding problem) is the one evaluated on the test set, so no test information is used for model selection. For HAR the validation set is 4 of the 21 training subjects per seed (a subject-wise split, so that windows of one person never appear in both training and validation), and the input normalisation statistics are computed from the training part only. `nn.RNN/LSTM/GRU` are used for training because they are faster; hand-written from-scratch cells are used for the unit tests and for the gradient analysis, where access to every hidden state is needed.

### 1.2 Verification of the RNN, LSTM and GRU cells

The from-scratch cells are used for the tests and the gradient analysis, and nn.RNN/LSTM/GRU for training. The reported parameter counts follow exactly from the model definitions with 9 input channels, hidden size h and 6 output classes. For the GRU, 3(h(9+h)+2h) + 6(h+1) = 3(64 x 73 + 128) + 390 = 14,790 at h = 64, and 4,326 at h = 32. For the LSTM the gate factor is 4 instead of 3, giving 4 x 4,800 + 390 = 19,590, and for the plain RNN the factor is 1, giving 5,190. This confirms that the models are implemented with one bias pair per gate block (the PyTorch convention) and that the fair-comparison table below is not affected by accidental differences in the classifier head.

**Tasks and metrics.** UCI HAR is a six-class activity-recognition task (walking, upstairs, downstairs, sitting, standing, laying) on 9-channel inertial windows of 128 time steps (2.56 s). Accuracy and macro-F1 are both reported; macro-F1 is used for model selection because it weights all six classes equally and is therefore less sensitive to the slight class imbalance and to the sitting/standing confusion that dominates the errors. The adding problem takes a sequence of length T with a random value and a binary marker in each step and asks the model to output the sum of the two marked values; its difficulty is controlled only by T, which makes it a clean probe of long-range memory. A model that ignores the input and predicts the mean reaches an MSE of 0.170, which is why 0.05 is used as the 'solved' threshold (well below the baseline, well above the best results).

| Component | Problem 1 | Problem 2 - Task A | Problem 2 - Task B |
|---|---|---|---|
| Model | RNN / LSTM / GRU, h = 64, linear head on last state | Pre-LN encoder, 2 layers, d = 128, 4 heads, CLS head | Pre-LN decoder, 4 layers, d = 128, causal mask |
| Input | 128 steps x 9 channels | 16 patches of 8 steps + CLS | 128 characters, vocab 65 |
| Optimiser | Adam 1e-3, clip 1.0 | AdamW 5e-4 | AdamW 1e-3, warm-up 100 + cosine |
| Budget | 20 (HAR) / 30 (adding) epochs | 20 epochs | 12 x 250 steps (ablations 4 x 250) |
| Selection | best val. macro-F1 / MSE | best val. macro-F1 | final / last-10% validation loss |

*Table 0: experimental set-up at a glance.*

### 1.3 Adding problem and experiments 5(a)-(c)

**Gradient flow across time (T = 100).**

The gradient norm $\|dL/dh_t\|$ was measured for every time step with the from-scratch cells. The table gives the ratio of the norm at t = 1 to the norm at t = 100, i.e. how much of the error signal that enters at the last step survives all the way back to the first step; a value near 1 means almost no decay, a value near 0 means the

early inputs are effectively invisible to learning.

| Model | Ratio before training | Ratio after training |
|---|---|---|
| RNN | 4.92e-05 | 1.15e-04 |
| LSTM | 4.23e-09 | 2.94e-01 |
| GRU | 9.54e-20 | 6.01e-01 |

*Table 1: gradient-norm ratio $||dL/dh_1|| / ||dL/dh_{100}||$.*

At initialisation all three models vanish, and the GRU vanishes the most (1e-19). This is expected: with default initialisation the gates are close to 0.5, so the effective per-step Jacobian has a norm well below 1 and the product over 100 steps collapses. After training the picture changes sharply for the gated models. The LSTM and GRU gradients are almost flat over time (ratios 0.29 and 0.60), which means the gates have learned to carry the error signal back to the early steps: the LSTM through an almost-open forget gate on the cell state, the GRU through an update gate that copies the previous state. The plain RNN stays at about 1e-4, so its early steps receive almost no gradient even after training. Training changed the ratio of the RNN by only a factor of about 2, whereas it changed the LSTM ratio by eight orders of magnitude and the GRU ratio by nineteen, so the improvement is a property of the gating mechanism rather than of optimisation alone.

**Why the gates help.** For a plain RNN, $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$, the gradient to step 1 is a product of 99 Jacobians, each containing $W_h^T$ and the derivative of tanh. If the largest singular value of that product is below 1 the signal decays exponentially; if it is above 1 it explodes, which is why gradient clipping at 1.0 is used for all models. In the LSTM the cell state is updated additively, $c_t = f_t * c_{t-1} + i_t * g_t$, so the Jacobian along the cell path is simply the forget gate $f_t$; when $f_t$ is close to 1 the gradient passes almost unchanged. The GRU has the analogous path $h_t = (1 - z_t) * h_{t-1} + z_t * h\tilde{}_t$, where a small update gate $z_t$ copies the old state. The measured post-training ratios of 0.29 and 0.60 are exactly what such near-identity paths would produce, and they also explain why the forget-gate bias is a natural variable to study: it sets how close to the identity the LSTM path starts.

**Sequence-length sweep and where each model fails.**

A run counts as solved if the test MSE is below 0.05; the constant (predict-the-mean) baseline reaches 0.170. Each cell is the mean over three seeds, with the number of solved seeds in brackets.

| T | RNN MSE (solved) | LSTM MSE (solved) | GRU MSE (solved) |
|---|---|---|---|
| 50 | 0.0241 (3/3) | 0.0046 (3/3) | 0.0015 (3/3) |
| 100 | 0.1225 (0/3) | 0.0662 (1/3) | 0.0021 (3/3) |
| 200 | 0.1678 (0/3) | 0.1684 (0/3) | 0.0031 (3/3) |
| 400 | 0.1538 (0/3) | 0.1535 (0/3) | 0.0027 (3/3) |

*Table 2: adding-problem test MSE against sequence length (30 epochs).*

The plain RNN already fails at T = 100 (0/3 seeds solved). The LSTM solves T = 50 comfortably, solves only one of three seeds at T = 100, and fails at T = 200, where its MSE sits at the baseline level (0.168) for all 30 epochs, meaning that it never found the signal. The GRU solves every length with MSE between 0.0015 and 0.0031, with no clear degradation from T = 50 to T = 400. In the T = 200 learning curves the GRU leaves the plateau at about epoch 13, whereas LSTM and RNN never leave it within 30 epochs. Two caveats apply: the T = 400 numbers of the failing models (0.153-0.154) are slightly below the T = 200 ones, but both are still at baseline level, so this is noise around 'not learned' and not a sign of partial learning; and the LSTM result is a statement about this fixed epoch budget and learning rate, not about the LSTM's ultimate capability (see the forget-bias ablation below).

**Forget-gate bias ablation (student ID ends in 9, odd; T = 200).** Initialising the LSTM forget-gate bias to 0 gives test MSE 0.1685 +- 0.0002 and initialising it to 1 gives 0.1684 +- 0.0004; both solve 0/3 seeds. The only visible effect is on the first few validation epochs, where bias 0 has the higher (worse) MSE, as expected because a forget gate that starts near 0.5 forgets half of the cell state at every step while a gate near 0.73 forgets less. However, a positive bias alone does not rescue training within 30 epochs at this length. This is consistent with the literature, which recommends a forget bias of about 1 as a helpful default but not as a fix

for very long dependencies; a longer schedule, a larger learning rate or a higher bias value would be the next things to try, but they were outside the fixed protocol of this assignment.

**1.4 HAR results: accuracy, parameters and time (hidden size 64)**

| Model | Test acc. | Macro-F1 | Params | s/epoch |
|---|---|---|---|---|
| RNN | 0.8422 +- 0.0116 | 0.8388 | 5,190 | 0.214 |
| LSTM | 0.8973 +- 0.0221 | 0.8973 | 19,590 | 0.206 |
| GRU | 0.8896 +- 0.0046 | 0.8888 | 14,790 | 0.179 |

*Table 3: HAR results at hidden size 64 (mean +- std over seeds 11, 22, 33).*

| Hidden | GRU acc | GRU params | GRU s/ep | LSTM acc | LSTM params | LSTM s/ep |
|---|---|---|---|---|---|---|
| 16 | 0.8646 | 1,398 | 0.194 | 0.8515 | 1,830 | 0.202 |
| 32 | 0.8884 | 4,326 | 0.193 | 0.8711 | 5,702 | 0.202 |
| 64 | 0.8896 | 14,790 | 0.194 | 0.8973 | 19,590 | 0.199 |
| 128 | 0.8961 | 54,150 | 0.241 | 0.9083 | 71,942 | 0.369 |

*Table 4: capacity sweep for GRU and LSTM.*

The gated models beat the plain RNN by about 5 accuracy points (0.84 against 0.89-0.90), at 2.9-3.8x the parameters of the RNN. The GRU is the cheapest per epoch at h = 64 (0.179 s) and has the lowest variance across seeds (std 0.0046), while the LSTM has a std of 0.009-0.025, so several of the differences between GRU and LSTM are within seed noise. In particular, the 0.0077 accuracy difference between LSTM-64 and GRU-64 is smaller than the LSTM's own standard deviation and should not be read as a real ranking. The capacity sweep shows different saturation behaviour: GRU accuracy saturates early (0.8884 at h = 32 against 0.8961 at h = 128, which is 12.5x the parameters), while the LSTM keeps improving and reaches the best accuracy overall, 0.9083 at h = 128, at 71,942 parameters and 0.369 s/epoch (1.8x the h = 64 time). At small sizes the GRU is clearly better (0.8646 against 0.8515 at h = 16 and 0.8884 against 0.8711 at h = 32), because it spends fewer parameters per hidden unit. The per-epoch time is almost flat up to h = 64 for every model, which indicates that on this GPU the cost at small sizes is dominated by kernel-launch and data-loading overhead and not by arithmetic.

**Accuracy per parameter and marginal gains.** Because the wearable question is about cost, Table 5 gives accuracy per 1,000 parameters and the marginal gain of each step up in hidden size. Both are computed directly from Tables 3 and 4.

| Model | Acc. per 1k params | Next step | Delta acc | Delta params |
|---|---|---|---|---|
| RNN-64 | 0.162 | - | - | - |
| GRU-16 | 0.618 | 16 -> 32 | +0.0238 | +2,928 |
| GRU-32 | 0.205 | 32 -> 64 | +0.0012 | +10,464 |
| GRU-64 | 0.060 | 64 -> 128 | +0.0065 | +39,360 |
| LSTM-16 | 0.465 | 16 -> 32 | +0.0196 | +3,872 |
| LSTM-32 | 0.153 | 32 -> 64 | +0.0262 | +13,888 |
| LSTM-64 | 0.046 | 64 -> 128 | +0.0110 | +52,352 |

*Table 5: accuracy per 1,000 parameters, and the gain from the next hidden-size step (GRU rows: 16 -> 32 -> 64 -> 128; LSTM rows likewise).*

The GRU's marginal gain collapses after h = 32 (+0.0012 for +10,464 parameters), whereas the LSTM still gains +0.0262 from 32 to 64 and +0.0110 from 64 to 128. The plain RNN has the highest accuracy per parameter simply because it is the smallest, but its absolute accuracy is 5 points lower, so this ratio alone is not a good selection criterion; it must be read together with the absolute accuracy.

**1.5 One-page analysis and wearable recommendation**

**Recommended: GRU with hidden size 32.** It reaches 0.8884 test accuracy with only 4,326 parameters (3.4x fewer than GRU-64 and 4.5x fewer than LSTM-64) at 0.193 s/epoch, within 0.0012 of GRU-64. A wearable has tight limits on memory, energy and latency, and the parameter count is the most direct proxy for all three. In addition, the GRU is the only model that solved the long-range adding task at all lengths (T up to 400), which matters for slowly varying activity context such as long periods of sitting or lying with short transitions in between. If memory allows, GRU-64 (0.8896, 14,790 parameters) is the safe choice. LSTM-128 is not recommended: its gain of about +0.01 accuracy over GRU-64 costs 71,942 parameters (4.9x more) and about twice the time, and the gain is of the same order as the seed noise. If the application were server-side and accuracy was the only criterion, LSTM-128 would be the best HAR model of Problem 1.

## 2. Problem 2 - Transformer from scratch

### 2.1 Setup, attention and encoder-block tests

The instructor's 3-token example was not available, so a hand-checkable example is used instead; the code and the hand calculation agree. `nn.MultiheadAttention` and `nn.Transformer*` are not used anywhere. *Task A* (HAR classifier): patch size 8 (16 patches plus a CLS token), 2 pre-LN encoder layers, d = 128, 4 heads, AdamW with learning rate 5e-4, 20 epochs, model selection on validation macro-F1. *Task B* (character-level causal language model on Tiny Shakespeare): 4 layers, d = 128, context 128, AdamW 1e-3 with a 100-step warm-up followed by cosine decay, 12 x 250 steps (ablations 4 x 250 steps), validation on the whole last 10% of the text in non-overlapping 128-character windows.

The attention and encoder-block tests use the hand-checkable example described above: the code output and the hand calculation agree. `nn.MultiheadAttention` and `nn.Transformer*` are not used for any result.

### 2.2 Parameter count derivation (Task A, no positional encoding)

With d = 128, 2 layers and a feed-forward width of 512 (4d): patch embedding 72 x 128 + 128 = 9,344; CLS token 128; per layer, attention 4(128 x 128 + 128) = 66,048, feed-forward (128 x 512 + 512) + (512 x 128 + 128) = 131,712 and two LayerNorms 4 x 128 = 512, i.e. 198,272, hence 396,544 for two layers; final LayerNorm 256; classifier 128 x 6 + 6 = 774. The total is 9,344 + 128 + 396,544 + 256 + 774 = **407,046**, matching the code. The learned positional table adds 256 x 128 = 32,768 parameters (maximum length 256), giving 439,814; sinusoidal encodings have no parameters.

### 2.3 Task A: HAR classification with a Transformer encoder (3 seeds)

**Architecture.** The 128-step window is cut into 16 non-overlapping patches of 8 steps (72 raw values each), linearly embedded to d = 128 and prefixed with a learnable CLS token; two pre-LN encoder layers (multi-head self-attention then a position-wise feed-forward block, each with a residual connection) follow, and the CLS output feeds a linear classifier. The attention is written out explicitly as $\text{softmax}(QK^T/\text{sqrt}(d_{head}))V$ with per-head projections, which is the part that the hand-checked 3-token example verifies against the code. Pre-LN is used because it trains stably without a long warm-up at this small scale.

| Positional encoding | Test acc. | Macro-F1 | Params |
|---|---|---|---|
| None | 0.9313 +- 0.0089 | 0.9313 | 407,046 |
| Sinusoidal | 0.9104 +- 0.0091 | 0.9106 | 407,046 |
| Learned | 0.9092 +- 0.0094 | 0.9090 | 439,814 |

*Table 6: positional-encoding ablation on HAR. Training time is about 0.47-0.49 s/epoch for all three.*

Positional information did not help here. The patch embedding of 8 raw samples already carries the local temporal order inside each patch, and the CLS-token classifier relies mostly on the overall signal statistics of the activity (for example the signal energy and the gravity direction) and not on the exact position of a pattern inside the 2.56-second window. Sinusoidal and learned encodings add an input that the model has to learn to ignore, which is a plausible reason for the slightly lower accuracy. The gap is 0.02 and about two standard deviations, so it is a modest effect and with three seeds it is suggestive rather than conclusive. The learned

encoding adds 32,768 parameters (439,814 - 407,046) for no gain, so on this dataset the cost-free no-encoding model is preferable. This should not be generalised: on tasks where the order of events matters (such as Task B) positional information is essential. Note also that the Transformer (0.93) is above the best Problem 1 model (0.8973, LSTM-64), but it has about 21x the parameters of LSTM-64 (407,046 against 19,590) and runs at roughly 2.3x the time per epoch (0.47-0.49 s against 0.206 s), so the comparison is not at equal cost.

### 2.4 Task B: learning curve and bits per character (Tiny Shakespeare, seed 11)

The corpus has 1,115,394 characters and a vocabulary of 65, so a uniform guess costs ln 65 = 4.174 nats per character. After 12 x 250 steps the final validation loss is 1.6753 nats = 2.4170 bits per character, i.e. the model removed about 60% of the uniform-guess uncertainty. This is well within the range expected for a small 4-layer, d = 128 model trained for only 3,000 steps; longer training or a larger model would be needed to approach the 1.0-1.3 bits/char level of large character models. Generated samples depend strongly on temperature: at 0.2 the text is repetitive (for example 'the shall son ... of the stand of the consul'), at 0.8 it is the most Shakespeare-like with the right line layout and some invented words, and at 1.5 it is noisy with broken words. Low temperature concentrates the distribution on the most likely characters and produces loops, while high temperature flattens the distribution and lets unlikely characters through; 0.8 is a reasonable trade-off between fidelity and diversity.

### 2.5 Heatmaps and ablation: number of attention heads (4 epochs, equal d = 128)

| Experiment | Val loss (nats) | Bits/char |
|---|---|---|
| 1 head (d_head 128) | 1.9703 | 2.8425 |
| 4 heads (d_head 32) | 1.9859 | 2.8651 |
| 8 heads (d_head 16) | 1.9952 | 2.8785 |

*Table 7: effect of the number of heads at fixed model width and a short 4 x 250-step budget.*

Fewer heads were slightly better in this short budget (1.9703 against 1.9952 nats), but the total spread is only 0.025 nats and comes from a single seed, so no strong conclusion can be drawn. A possible explanation is that with d = 128 each head of the 8-head model has only 16 dimensions, which limits what one head can represent, and that the benefit of multiple heads (several different attention patterns) appears mainly after longer training. The last-layer attention heatmaps show that heads do specialise: head 1 has a strong near-diagonal, local pattern, while head 2 attends to a few earlier positions such as the newline and colon characters, which mark the start of a speaker line. This is a qualitative observation from one layer and one sample.

### 2.6 Ablation: removing the causal mask

Without the mask, training loss collapses to 0.0773 and validation loss to 0.0246 nats (0.035 bits/char), because position t can read the target at t + 1 through attention, so the task becomes a trivial copy operation. This is a leak and not a better model: when the same network is evaluated *with* the causal mask, it gets 7.0032 nats (10.10 bits/char), which is worse than the uniform guess of 4.174, and its generation is degenerate ('ssssss sesese'). The correctly masked model reaches 1.9859 nats in the same ablation setting and generates plausible text. The experiment shows why a very low loss must always be checked for information leakage before it is reported.

### 2.7 Ablation: context length, time and memory scaling

| Context | Step time (s) | Peak memory (MB) | Time ratio vs 64 | n^2 ratio |
|---|---|---|---|---|
| 64 | 0.0104 | 86.23 | 1.00 | 1 |
| 128 | 0.0102 | 133.85 | 0.98 | 4 |
| 256 | 0.0188 | 267.46 | 1.81 | 16 |
| 512 | 0.0532 | 712.46 | 5.11 | 64 |

*Table 8: training-step cost against context length.*

The attention score matrix grows as n^2, from 262,144 to 16,777,216 entries per layer between context 64 and 512 (a factor of 64). Peak memory rises from 86 MB to 712 MB (8.3x for 8x the context), which is far below 64x because the model parameters, optimiser state and the activations of the linear layers (which scale linearly in n) form a fixed or linear part. The measured time has a log-log slope of 0.79, well below the theoretical 2: at small contexts the GPU is under-utilised and fixed launch overhead dominates (context 64 and 128 take the same time). The cost becomes clearly super-linear at 256-512 (5.11x time for 8x the context), and the n^2 term dominates at longer contexts. This is why long-context models use memory-efficient attention variants.

## 3. Analysis, limitations and conclusions

**Main findings.** (i) Gating is what makes long-range credit assignment possible: gradient ratios after training are 0.29 (LSTM) and 0.60 (GRU) against 1e-4 for the RNN, and only the GRU solves the adding task up to T = 400 within the 30-epoch budget. (ii) On HAR the gated models are about 5 points above the plain RNN, the GRU gives the best accuracy per parameter, and the LSTM gives the best absolute accuracy at h = 128. (iii) A from-scratch Transformer gives the best HAR accuracy (0.9313) but needs no positional encoding on this task and costs about 21x the parameters of LSTM-64. (iv) The causal mask is essential for language modelling, and attention cost grows quadratically with context once the GPU is saturated.

**Limitations.** Problem 1 uses three seeds and Problem 2 Task A uses three seeds, so differences smaller than about two standard deviations are not reliable; Task B and all of its ablations use a single seed (11). The LSTM failure at T = 200 and the null result of the forget-bias ablation are tied to the fixed budget of 30 epochs and learning rate 1e-3 and should not be read as a limit of the LSTM in general. The head and positional-encoding comparisons were run with short schedules, so rankings could change with longer training. Timings were measured on a single GPU and include data-loading overhead, which dominates at small model sizes.

**Reproducibility.** All experiments use seeds 11, 22 and 33 (Task B: 11), the hyper-parameters listed in Section 1, and the selection rule 'best validation epoch'. The numbers in this report are taken from the notebook outputs and from results.csv, and the code and the README describe how to rerun every table.

**Practical guidance.** For a short-context sensor task such as HAR a small GRU is the best default: it trains quickly, is stable across seeds and is cheap to deploy. If the activity context spans hundreds of steps, the adding-problem results say that the GRU should be preferred over the LSTM under a fixed small training budget, and over the plain RNN in every case. If accuracy per parameter is not a concern and the data are static windows, a small Transformer is the strongest of the three families on HAR in this study, and it is also the only one that scales naturally to the language-modelling task. A fair choice between families should always include the cost columns (parameters, time, memory) and not only accuracy.

**Possible extensions.** The ablations point to cheap follow-ups that were outside the fixed protocol: a longer schedule or a learning-rate sweep for the LSTM at T = 200 with forget bias 1, a forget bias of 2-3, more seeds for the Task B ablations, a head ablation with the same number of total training steps as the main run, and a comparison of the Transformer and GRU at equal parameter budgets on HAR.

## 4. AI-use statement and references

### AI-use statement

An AI assistant (Claude, by Anthropic) was used to help correct and structure the notebooks and to help draft and edit the text of this report. All experiments were run by the student, all numbers in the report come from the student's own runs, and the student reviewed and takes responsibility for the code, the results and the interpretation.

### References

[1] S. Hochreiter and J. Schmidhuber, 'Long short-term memory', *Neural Computation* 9(8), 1997.

[2] K. Cho et al., 'Learning phrase representations using RNN encoder-decoder for statistical machine translation', EMNLP 2014.

[3] R. Pascanu, T. Mikolov and Y. Bengio, 'On the difficulty of training recurrent neural networks', ICML 2013.

[4] R. Jozefowicz, W. Zaremba and I. Sutskever, 'An empirical exploration of recurrent network architectures', ICML 2015.

[5] D. Anguita et al., 'A public domain dataset for human activity recognition using smartphones', ESANN 2013 (UCI HAR dataset).

[6] A. Vaswani et al., 'Attention is all you need', NeurIPS 2017.

[7] I. Loshchilov and F. Hutter, 'Decoupled weight decay regularization', ICLR 2019 (AdamW).

[8] R. Xiong et al., 'On layer normalization in the Transformer architecture', ICML 2020 (pre-LN).

[9] A. Karpathy, 'The unreasonable effectiveness of recurrent neural networks', 2015 (Tiny Shakespeare corpus).